

EG-RSA: Memory-Reflective Reward Schema Adaptation without Oracle Reward Feedback

Anonymous Authors
Draft for local editing

Abstract—Reward design remains a central bottleneck in reinforcement learning. Recent large language model based methods can generate reward functions from task descriptions and environment code, but they often rely on weak iteration, task-specific feedback, human correction, or oracle task scores to select and improve rewards. This paper presents EG-RSA, a memory-reflective reward schema adaptation framework that iteratively evolves reward functions without using official environment rewards as search feedback. EG-RSA represents rewards as structured, editable schemas rather than unrestricted Python code, builds non-oracle feedback from rollout behavior and reward component attribution, and uses a ReflectionAgent to reason over three levels of memory before an EditAgent proposes schema-level reward modifications. A SchemaEngine canonicalizes, validates, and commits edits to ensure safe execution and stable iteration.

I. INTRODUCTION

Reinforcement learning (RL) depends critically on the reward function used to guide policy optimization. A well-designed reward can accelerate learning and align the policy with the intended task, whereas an incomplete or misspecified reward may induce sparse exploration, unstable training, reward hacking, or behaviors that maximize the designed signal while failing the real task objective. Designing such rewards typically requires expert knowledge of the environment dynamics, the task semantics, and the failure modes of the learner.

Large language models (LLMs) provide a promising alternative for reward design because they can interpret task descriptions, reason over environment code, and synthesize executable programs. Recent work such as Eureka demonstrates that coding LLMs can generate reward functions that support reinforcement learning across diverse control tasks [1]. Other methods, including Text2Reward and language-model-based reward design, further show that natural language can be used to specify desired behavior or shape dense rewards [2], [3]. These works suggest a new direction: using LLMs not merely as policy planners, but as reward designers.

However, LLM-based reward generation still faces three limitations. First, a reward generated in a single pass is often insufficient. Reward functions must be evaluated through training and then revised when they produce poor exploration, unstable learning, or undesired shortcuts. Second, feedback used for reward improvement is often entangled with official task rewards, human correction, or manually designed success metrics. Such feedback can be effective, but it weakens the claim that the reward is discovered independently of the

task’s oracle reward. Third, direct Python reward generation is flexible but difficult to validate, edit, attribute, and reuse across iterations.

This paper proposes EG-RSA: a memory-reflective reward schema adaptation framework for LLM-driven reward self-evolution without oracle reward feedback. Instead of asking an LLM to repeatedly output unrestricted Python reward code, EG-RSA represents a reward function as a structured reward schema. A schema consists of formula-based reward components, event predicates, weights, clips, and semantic roles. This representation preserves expressiveness while enabling canonicalization, validation, component attribution, and localized edits.

The feedback mechanism in EG-RSA is deliberately non-oracle. During policy training, the environment reward is replaced by the schema reward. The official reward, if available, is not used to guide reward search. After training, EG-RSA builds feedback from rollout trajectories, reward component attribution, trajectory summaries, and behavior-level diagnostics. This feedback tells the system whether the current reward is dominated by a single component, whether the policy exploits sparse events, whether progress is insufficient, or whether the observed behavior mismatches the task description.

A key design of EG-RSA is memory-reflective iteration. The system maintains three levels of memory: raw memory cards that record exact edit and outcome traces, distilled lesson cards that summarize reusable experience, and outcome lessons that compare before/after effects of committed edits. A ReflectionAgent examines these memory layers and decides which evidence to reuse, ignore, or treat as conflicting. An EditAgent then turns the reflection strategy into a concrete schema-level edit plan. Finally, a SchemaEngine validates the edit plan, applies the committed edits transactionally, canonicalizes the next schema, and passes it to the next training iteration.

The main contributions of this work are:

- We introduce a non-oracle feedback mechanism for LLM-driven reward search. The method improves rewards using rollout behavior, component attribution, and task-proxy diagnostics rather than official environment rewards.
- We propose a memory-reflective reward iteration mechanism. The ReflectionAgent selectively reasons over raw memory, distilled lessons, and measured outcome lessons before reward editing.
- We formulate reward generation as schema-level self-evolution. The reward schema acts as a safe, editable in-

intermediate representation that supports canonicalization, validation, attribution, and iterative modification.

II. RELATED WORK

A. Reward Design and Reward Shaping

Reward design is a longstanding challenge in reinforcement learning. Manual reward shaping can improve sample efficiency and stabilize learning, but it requires substantial task expertise and may introduce unintended optimization targets. A shaped reward that appears reasonable at the component level may still lead to reward hacking if the agent discovers shortcuts that exploit the designed signal. Preference-based reinforcement learning addresses reward specification by learning from human comparisons rather than fixed reward functions [4]. In contrast, EG-RSA does not learn a reward model from human labels. It uses LLMs to generate and edit reward schemas, while relying on non-oracle rollout feedback to guide iteration.

B. LLM-Based Reward Generation

LLMs have recently been used as reward designers. Reward Design with Language Models explores using language models as proxy reward functions based on textual descriptions [3]. Text2Reward generates dense reward code from natural language goals and compact environment representations, with optional human feedback for refinement [2]. Eureka demonstrates that coding LLMs can synthesize reward functions from environment code and iteratively improve reward programs, achieving strong performance across a broad set of RL environments [1].

EG-RSA is closest in spirit to this line of work, but differs in three ways. First, it focuses on reward improvement without using official environment rewards as the search feedback. Second, it represents rewards as structured schemas rather than free-form Python programs, enabling validation, attribution, and transactional edits. Third, it introduces memory-reflective reward iteration, where an LLM explicitly reasons over multiple memory layers before proposing the next reward modification.

C. LLM Agents with Memory and Reflection

LLM agents often combine reasoning, action, memory, and feedback. ReAct interleaves reasoning traces and environment actions, showing that language models can improve decision-making by combining thought and external interaction [5]. Reflexion introduces verbal reinforcement learning, where agents reflect on feedback and store reflective text in memory to improve future trials without updating model weights [6]. Voyager uses an automatic curriculum, a skill library, and iterative prompting to support open-ended embodied learning in Minecraft [7]. Generative Agents use memory, reflection, and planning to produce believable long-horizon behavior in interactive simulations [8].

EG-RSA adopts the insight that memory and reflection can improve LLM-based agents, but applies it to reward function search. The memory in EG-RSA is not merely a record of past

conversations or actions. It is organized around reward edits and their measured training outcomes. The ReflectionAgent must decide whether raw edit traces, distilled lessons, or outcome lessons are relevant to the current reward failure before the EditAgent modifies the schema.

D. Reward Hacking and Non-Oracle Feedback

Reward hacking occurs when an agent optimizes the specified reward while deviating from the designer’s intended objective. This issue is especially important in automatic reward design, where an LLM-generated reward may contain exploitable shortcuts. Many approaches detect reward quality using explicit success metrics, human feedback, or oracle evaluation. While useful, such signals may leak task-specific objectives into the reward search process.

EG-RSA separates reward search from official reward feedback. The official environment reward is not used by the reward editor or memory reflection process. Instead, the system constructs feedback from rollout trajectories, reward component attribution, event activation patterns, and behavior diagnostics. This design makes the feedback weaker than oracle task scores, but more aligned with the goal of autonomous reward self-improvement.

E. Structured Reward Programs and Safe Editing

Direct reward-code generation is expressive but difficult to control. A generated Python function may contain unsafe operations, hidden state, inconsistent scaling, or logic that cannot be locally edited. EG-RSA therefore uses a reward schema as an intermediate representation. Each schema contains typed components and event rules, and reward edits are expressed as constrained operators such as replacing a formula, changing a weight, adding a conditional formula component, or adding an event predicate. This structure enables safe evaluation, component attribution, and memory alignment across iterations.

III. METHOD

A. Problem Formulation

We consider an RL environment with observations o_t , actions a_t , transition dynamics, and an optional official environment reward r_t^{env} . The objective of EG-RSA is to automatically construct and iteratively improve a reward function $R_S(o_t, a_t)$ represented by a reward schema S . During reward search, the policy is trained using R_S rather than r_t^{env} . If the official environment reward exists, it is reserved for post-hoc reporting and is not provided to the reward search agents, memory system, or edit decisions.

The intended input to the system is minimal: a task description and an environment source or step function. The environment source is used to derive an internal interface, including available observation and action variables. This interface is not treated as a human-designed reward specification; it only constrains the variables and functions that can appear in generated reward formulas.

B. Reward Schema Representation

A reward schema S is a structured representation of a reward function. It contains a set of reward components $\mathcal{C}$ and event rules $\mathcal{E}$. Each component $c_i \in \mathcal{C}$ has a name, type, weight w_i , formula or condition, optional clipping range, and semantic role. Each event rule $e_j \in \mathcal{E}$ defines a predicate over primitive variables and may trigger a sparse one-time reward.

The schema reward is computed as:

$$R_S(o_t, a_t) = \sum_{c_i \in \mathcal{C}} w_i c_i(o_t, a_t) + \sum_{e_j \in \mathcal{E}} e_j(o_t, a_t). \quad (1)$$

The schema representation has four advantages. First, it avoids executing unrestricted LLM-generated reward code. Second, it supports local editing: the LLM can replace a formula, change a weight, add a component, or modify an event predicate. Third, it enables component-level attribution after training. Fourth, it provides a stable object for memory: the system can record which schema edit caused which outcome.

C. Bootstrap Schema Generation

At the beginning of a run, a BootstrapAgent generates an initial reward schema from the task description and internal environment interface. The initial schema typically contains dense guidance components, stability or safety components, control cost components, and sparse event predicates. The raw LLM output is not directly executed. It is passed through the SchemaEngine, which canonicalizes aliases, enforces sign conventions for control costs, validates safe formulas, and converts the result into a canonical schema.

D. Non-Oracle Feedback Builder

After training a policy with the current schema, EG-RSA collects trajectories and constructs a feedback report. The feedback report does not use the official environment reward. It is built from reward component attribution, trajectory summaries, task-proxy diagnostics, reward hacking signals, and outcome comparisons. This feedback provides the evidence used by the reflection and editing stages. The central design principle is that reward search should not depend on oracle task scores.

E. Three-Level Memory

EG-RSA maintains three memory layers. Raw memory cards store exact edit traces, schema snapshots, diagnostics, and pending outcomes. Distilled lesson cards compress prior runs into reusable lessons, such as edits that often failed or conditions under which coupled rebalancing was useful. Outcome lessons store measured before/after effects of committed edits, making them the most causally grounded memory layer. Unlike simple retrieval-augmented prompting, memory is not automatically trusted. The ReflectionAgent explicitly decides which memory layers to use, which to ignore, and how to resolve conflicts.

F. ReflectionAgent

The ReflectionAgent receives the current schema, feedback report, and retrieved memory. It does not output executable reward edits. Instead, it produces a reflection report containing a failure assessment, a memory assessment, a memory-use policy over the three memory layers, and a recommended search strategy such as single edit, coupled rebalancing, structural search, continue training, or early stop.

G. EditAgent

The EditAgent, or reward generation agent, converts the reflection report into a concrete schema edit plan. It may propose edits such as increasing or decreasing a component weight, replacing a component formula, replacing an event condition, adding a formula component, adding a conditional formula component, adding an event predicate, or adding a control-cost component. The EditAgent does not write arbitrary Python code. It outputs a constrained JSON edit plan.

H. SchemaEngine: Canonicalization, Validation, and Transition

The SchemaEngine is responsible for safe schema execution. It canonicalizes raw LLM output into a standard schema language, validates that formulas use allowed variables and functions, applies only committed edits to the next schema, and checks that the training wrapper and compiled reward artifact implement the same reward semantics. A key distinction is that an accepted edit is not necessarily a committed edit. Only committed edits change the next reward schema.

I. Iterative Reward Self-Evolution

The iterative loop of EG-RSA consists of: generating an initial schema, training a policy with the current schema reward, collecting trajectories, building non-oracle feedback, retrieving memory, generating a reflection report, producing an edit plan, committing a canonical next schema, and storing the edit outcome back into memory. This loop implements memory-reflective reward self-evolution without using official environment reward as search feedback.

REFERENCES

- [1] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," *arXiv preprint arXiv:2310.12931*, 2023.
- [2] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, and T. Yu, "Text2reward: Reward shaping with language models for reinforcement learning," *arXiv preprint arXiv:2309.11489*, 2023.
- [3] M. Kwon, S. M. Xie, K. Bullard, and D. Sadigh, "Reward design with language models," *arXiv preprint arXiv:2303.00001*, 2023.
- [4] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," in *Advances in Neural Information Processing Systems*, 2017.
- [5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.
- [6] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *arXiv preprint arXiv:2303.11366*, 2023.

- [7] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.
- [8] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," *arXiv preprint arXiv:2304.03442*, 2023.